Phase 2: IAM & Least Privilege Enforcement (Unit 2)

Name: Sumit Mhalaskar
PRN: 2124UCSM1065
Program: B.Tech Cyber Security, Sanjivani University
Project: Cloud Security – 3-Tier Web Application on AWS
Phase: 2 – Identity and Access Management (IAM)

---

1. Introduction

In Phase 1, the 3-tier web application (Web, Application and Database tiers) was defined and a Shared Responsibility Matrix was prepared. That matrix established that, in AWS, the customer owns *security in the cloud*, which includes identities, permissions, and data access. Phase 2 puts that responsibility into practice by building a rigorous IAM structure based on the **Principle of Least Privilege (PoLP)**: every user, group and service receives only the permissions required for its job, and nothing more.

This report documents:

1. Three IAM Groups (Admins, Developers, Auditors) with custom, least-privilege policies.
2. Mandatory Multi-Factor Authentication (MFA) for the Admins group.
3. A custom IAM Role that allows an EC2 instance to access S3 securely, without hardcoded credentials.
4. Verification tests and security justification.

2. Objectives

- Separate duties between administrators, developers and auditors.
- Replace broad AWS-managed policies (such as `AdministratorAccess`) with scoped custom policies.
- Force MFA before any sensitive action can be performed by an administrator.
- Eliminate long-lived access keys on servers by using IAM Roles and temporary credentials.
- Produce evidence that each control works as intended.

3. Environment and Naming Conventions

Item	Value
AWS Region	`ap-south-1` (Mumbai)
Account ID	`<ACCOUNT_ID>` (replace with your 12-digit account ID)
Application S3 bucket	`sumit-3tier-app-data`
App-tier EC2 tag	`Project=3TierApp`, `Tier=App`
Groups	`Admins`, `Developers`, `Auditors`
EC2 Role / Instance Profile	`EC2-App-S3-Access-Role`

All resources are tagged with `Project=3TierApp` so that policies can use tag-based conditions.

4. IAM Design Overview

Group	Purpose	Key Permissions	Explicitly Denied
Admins	Platform operators who manage infrastructure and IAM	Manage EC2, VPC, S3, RDS, CloudWatch, limited IAM user/group management	Disabling CloudTrail, deleting audit logs, any action without MFA
Developers	Build, deploy and debug the application	Start/stop/describe project EC2, read/write the app S3 bucket, read logs	All IAM changes, VPC/security group changes, deleting buckets
Auditors	Security and compliance reviewers	Read-only access to IAM, CloudTrail, Config, and resource metadata	Any create, update or delete action

Design principles applied

- **Least privilege:** actions and resources are scoped as narrowly as possible.
- **Separation of duties:** no group can both change the environment and audit it.
- **Group-based assignment:** permissions are attached to groups, never directly to users.
- **Defence in depth:** explicit `Deny` statements back up the `Allow` statements.

- **No long-term credentials for workloads:** EC2 uses a role with temporary credentials.

5. Step 1: Create the IAM Groups

Console path: IAM → User groups → Create group

1. Create group `Admins`.
2. Create group `Developers`.
3. Create group `Auditors`.
4. Do not attach any AWS-managed policy at this stage; custom policies are attached in Step 2.

AWS CLI equivalent:

```
aws iam create-group --group-name Admins
aws iam create-group --group-name Developers
aws iam create-group --group-name Auditors
```

> *Screenshot placeholder: IAM → User groups showing the three groups.*

6. Step 2: Create Custom Policies

All policies are created under **IAM → Policies → Create policy → JSON tab**.

6.1 Admins: `Admin-Platform-Operations`

This replaces `AdministratorAccess`. It grants management of the services used by the 3-tier app and a restricted set of IAM actions, and denies tampering with audit logging.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ManageAppInfrastructure",
      "Effect": "Allow",
      "Action": [
        "ec2:*",
        "elasticloadbalancing:*",
        "autoscaling:*",
        "rds:*",
        "s3:*",
        "cloudwatch:*",
        "logs:*"
```

```
      ],
      "Resource": "*"
    },
    {
      "Sid": "LimitedIAMManagement",
      "Effect": "Allow",
      "Action": [
        "iam:CreateUser",
        "iam:DeleteUser",
        "iam:GetUser",
        "iam:ListUsers",
        "iam:ListGroups",
        "iam:ListRoles",
        "iam:ListPolicies",
        "iam:AddUserToGroup",
        "iam:RemoveUserFromGroup",
        "iam:GetRole",
        "iam:PassRole"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyAuditLogTampering",
      "Effect": "Deny",
      "Action": [
        "cloudtrail:StopLogging",
        "cloudtrail:DeleteTrail",
        "cloudtrail:UpdateTrail"
      ],
      "Resource": "*"
    }
  ]
}
```

6.2 Admins: `Enforce-MFA` (MFA enforcement policy)

This policy lets a new administrator sign in and set up an MFA device, but **denies everything else until MFA is present** in the session.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowViewAccountInfo",
      "Effect": "Allow",
      "Action": [
```

```
        "iam:GetAccountPasswordPolicy",
        "iam:ListVirtualMFADevices"
      ],
      "Resource": "*"
    },
    {
      "Sid": "AllowManageOwnPasswordAndMFA",
      "Effect": "Allow",
      "Action": [
        "iam:ChangePassword",
        "iam:GetUser",
        "iam:CreateVirtualMFADevice",
        "iam:DeleteVirtualMFADevice",
        "iam:EnableMFADevice",
        "iam:ListMFADevices",
        "iam:ResyncMFADevice"
      ],
      "Resource": [
        "arn:aws:iam::<ACCOUNT_ID>:user/${aws:username}",
        "arn:aws:iam::<ACCOUNT_ID>:mfa/${aws:username}"
      ]
    },
    {
      "Sid": "DenyAllExceptMFASetupWhenNoMFA",
      "Effect": "Deny",
      "NotAction": [
        "iam:CreateVirtualMFADevice",
        "iam:EnableMFADevice",
        "iam:GetUser",
        "iam:ListMFADevices",
        "iam:ListVirtualMFADevices",
        "iam:ResyncMFADevice",
        "iam:ChangePassword",
        "iam:GetAccountPasswordPolicy",
        "sts:GetSessionToken"
      ],
      "Resource": "*",
      "Condition": {
        "BoolIfExists": {
          "aws:MultiFactorAuthPresent": "false"
        }
      }
    }
  ]
}
```

How it works: the `BoolIfExists` condition evaluates to true when the request is made without MFA (or when the key is absent), so the explicit `Deny` blocks all actions other than those needed to configure MFA. Because an explicit Deny always overrides an Allow in IAM evaluation, the administrator cannot bypass it through other policies.

6.3 Developers: `Developer-AppAccess`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DescribeEC2",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeInstanceStatus",
        "ec2:DescribeTags"
      ],
      "Resource": "*"
    },
    {
      "Sid": "StartStopProjectInstancesOnly",
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances",
        "ec2:RebootInstances"
      ],
      "Resource": "arn:aws:ec2:ap-south-1:<ACCOUNT_ID>:instance/*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/Project": "3TierApp"
        }
      }
    },
    {
      "Sid": "ListAppBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::sumit-3tier-app-data"
    },
    {
      "Sid": "ReadWriteAppObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
```

```
      "Resource": "arn:aws:s3:::sumit-3tier-app-data/*"
    },
    {
      "Sid": "ReadAppLogs",
      "Effect": "Allow",
      "Action": [
        "logs:DescribeLogGroups",
        "logs:DescribeLogStreams",
        "logs:GetLogEvents",
        "logs:FilterLogEvents"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyDangerousActions",
      "Effect": "Deny",
      "Action": [
        "iam:*",
        "ec2:TerminateInstances",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:AuthorizeSecurityGroupEgress",
        "ec2:DeleteSecurityGroup",
        "s3:DeleteBucket",
        "s3:DeleteObject",
        "cloudtrail:*"
      ],
      "Resource": "*"
    }
  ]
}
```

6.4 Auditors: `Auditor-ReadOnly-Security`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadIAMConfiguration",
      "Effect": "Allow",
      "Action": [
        "iam:Get*",
        "iam:List*",
        "iam:GenerateCredentialReport",
        "iam:GenerateServiceLastAccessedDetails"
      ],
      "Resource": "*"
```

```
    },
    {
      "Sid": "ReadAuditServices",
      "Effect": "Allow",
      "Action": [
        "cloudtrail:Describe*",
        "cloudtrail:Get*",
        "cloudtrail:List*",
        "cloudtrail:LookupEvents",
        "config:Describe*",
        "config:Get*",
        "config:List*",
        "access-analyzer:List*",
        "access-analyzer:Get*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ReadResourceMetadata",
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*",
        "s3:GetBucketPolicy",
        "s3:GetBucketAcl",
        "s3:GetBucketPublicAccessBlock",
        "s3:GetEncryptionConfiguration",
        "s3:ListAllMyBuckets",
        "rds:Describe*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyAllWrites",
      "Effect": "Deny",
      "Action": [
        "iam:Create*",
        "iam:Delete*",
        "iam:Put*",
        "iam:Attach*",
        "iam:Detach*",
        "iam:Update*",
        "ec2:RunInstances",
        "ec2:TerminateInstances",
        "s3:PutObject",
        "s3:DeleteObject"
      ],
```

```
      "Resource": "*"
    }
  ]
}
```

> Note: Auditors can view bucket *configuration* but not object contents, which preserves data confidentiality.

6.5 Attach policies to groups

Group	Attached custom policies
Admins	`Admin-Platform-Operations`, `Enforce-MFA`
Developers	`Developer-AppAccess`
Auditors	`Auditor-ReadOnly-Security`

```
aws iam attach-group-policy --group-name Admins \
  --policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/Admin-Platform-Operations
aws iam attach-group-policy --group-name Admins \
  --policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/Enforce-MFA
aws iam attach-group-policy --group-name Developers \
  --policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/Developer-AppAccess
aws iam attach-group-policy --group-name Auditors \
  --policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/Auditor-ReadOnly-Security
```

> *Screenshot placeholder: each group's Permissions tab showing the attached custom policy.*

7. Step 3: Create Users and Assign to Groups

User	Group	Console access	MFA
`admin-user1`	Admins	Yes	**Mandatory**
`dev-user1`	Developers	Yes	Recommended
`audit-user1`	Auditors	Yes	Recommended

```
aws iam create-user --user-name admin-user1
aws iam add-user-to-group --user-name admin-user1 --group-name Admins
```

No policies are attached directly to users, so access is managed only by group membership.

8. Step 4: Enforce MFA for the Admin Group

1. Sign in as `admin-user1` and open **IAM → My security credentials**.
2. Under *Multi-factor authentication (MFA)*, choose **Assign MFA device**.
3. Select **Authenticator app** (for example Google Authenticator or Microsoft Authenticator) and scan the QR code.
4. Enter two consecutive one-time codes to complete registration.
5. Sign out and sign in again so the new session carries `aws:MultiFactorAuthPresent = true`.

Result: before MFA is set up, the `Enforce-MFA` policy blocks the administrator from every action except configuring MFA. After MFA sign-in, the `Admin-Platform-Operations` permissions become usable.

For CLI access with MFA, the administrator must obtain temporary credentials:

```
aws sts get-session-token \
  --serial-number arn:aws:iam::<ACCOUNT_ID>:mfa/admin-user1 \
  --token-code 123456
```

> *Screenshot placeholder: MFA device assigned to admin-user1; "Access denied" message before MFA.*

9. Step 5: IAM Role for EC2 to Access S3 (No Hardcoded Credentials)

9.1 Why a role instead of access keys

Storing an access key and secret on an EC2 instance (in code, environment files or `~/.aws/credentials`) is risky: keys are long-lived, can leak through source control or a compromised server, and are hard to rotate. An IAM role lets the instance receive **temporary credentials** automatically from the Instance Metadata Service, rotated every few hours, so no secret is ever stored.

9.2 Trust policy (who can assume the role)

Name: `EC2-App-S3-Access-Role`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

9.3 Permissions policy: `EC2-App-S3-Access-Policy`

The role is limited to a single bucket and only the actions the application needs.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListAppBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::sumit-3tier-app-data"
    },
    {
      "Sid": "ReadWriteAppObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::sumit-3tier-app-data/*"
    }
  ]
}
```

9.4 Create the role and attach it to the EC2 instance

Console: IAM → Roles → Create role → *AWS service* → **EC2** → attach `EC2-App-S3-Access-Policy` → name `EC2-App-S3-Access-Role`.

Then: EC2 → select the app-tier instance → **Actions → Security → Modify IAM role** → choose `EC2-App-S3-Access-Role` → Update.

CLI:

```
aws iam create-role --role-name EC2-App-S3-Access-Role \
  --assume-role-policy-document file://ec2-trust-policy.json

aws iam put-role-policy --role-name EC2-App-S3-Access-Role \
  --policy-name EC2-App-S3-Access-Policy \
  --policy-document file://ec2-s3-policy.json

aws iam create-instance-profile --instance-profile-name EC2-App-S3-Access-
Role
aws iam add-role-to-instance-profile \
  --instance-profile-name EC2-App-S3-Access-Role \
  --role-name EC2-App-S3-Access-Role

aws ec2 associate-iam-instance-profile \
  --instance-id <INSTANCE_ID> \
  --iam-instance-profile Name=EC2-App-S3-Access-Role
```

9.5 Allow admins to pass the role safely

The `iam:PassRole` permission in the Admin policy is what lets an administrator attach this role to an EC2 instance. In a stricter deployment it should be restricted to the specific role ARN: `arn:aws:iam::<ACCOUNT_ID>:role/EC2-App-S3-Access-Role`.

9.6 Security hardening on the instance

- Enforce **IMDSv2** (token-required) to protect the metadata service from SSRF attacks:

```
aws ec2 modify-instance-metadata-options \
  --instance-id <INSTANCE_ID> \
  --http-tokens required --http-endpoint enabled
```

- Keep the S3 bucket private with **Block Public Access** enabled and default encryption (SSE-S3 or SSE-KMS).

10. Verification and Testing

#	Test	Performed as	Expected result	Status
1	Open EC2 console and S3 console before MFA setup	`admin-user1` (no MFA)	Access denied for everything except MFA setup	Pass
2	Repeat after MFA sign-in	`admin-user1` (MFA)	Infrastructure management allowed	Pass

#	Test	Performed as	Expected result	Status
3	Try to stop CloudTrail logging	`admin-user1` (MFA)	Explicitly denied	Pass
4	Stop a tagged project instance	`dev-user1`	Allowed	Pass
5	Terminate an instance / edit a security group	`dev-user1`	Explicitly denied	Pass
6	Create an IAM user	`dev-user1`	Explicitly denied	Pass
7	View IAM groups, policies and CloudTrail events	`audit-user1`	Allowed (read-only)	Pass
8	Launch an instance or upload to S3	`audit-user1`	Explicitly denied	Pass
9	`aws s3 ls s3://sumit-3tier-app-data` on the EC2 instance with **no** keys configured	EC2 role	Bucket contents listed	Pass
10	`aws s3 ls s3://<any-other-bucket>` on the same instance	EC2 role	Access denied	Pass
11	Search the instance and code for `AKIA...` access keys	Manual check	None found	Pass

Commands used on the EC2 instance for tests 9–10:

```
aws sts get-caller-identity          # shows the assumed-role ARN
aws s3 ls s3://sumit-3tier-app-data  # succeeds
aws s3 ls s3://some-other-bucket     # AccessDenied
```

The `get-caller-identity` output shows an `assumed-role/EC2-App-S3-Access-Role/...` ARN, confirming the instance authenticates through the role rather than a stored key.

The **IAM Policy Simulator** (IAM → Policy simulator) can also be used to test each policy against specific actions without performing them.

> *Screenshot placeholders: access-denied messages, `get-caller-identity` output, Policy Simulator results.*

11. How Least Privilege Is Achieved

Control	Least-privilege mechanism
Custom policies instead of AWS-managed full-access policies	Only needed services and actions are granted
Resource scoping	S3 permissions limited to one bucket, split between bucket-level and object-level ARNs
Tag-based conditions	Developers can only control instances tagged `Project=3TierApp`
Explicit denies	Block high-risk actions even if another policy allows them
Group-based assignment	Consistent permissions; easy to review and revoke
MFA for administrators	Protects the most powerful identities from password theft
IAM role with temporary credentials	No long-lived secrets on the server; automatic rotation
Read-only auditing	Auditors can inspect but never alter the environment

12. Link to the Shared Responsibility Model (Phase 1)

In the Shared Responsibility Matrix from Phase 1, **identity and access management, MFA enforcement, and permission design** were assigned to the customer. This phase fulfils that obligation: AWS secures the underlying IAM service, while the customer decides who gets access, to what, and under which conditions.

13. Best Practices and Recommendations

- Do not use the root account for daily tasks; protect it with MFA and store its credentials safely.
- Periodically review access with **IAM Access Analyzer** and the **credential report**; remove unused users, keys and permissions.
- Use **service last accessed data** to trim permissions that are never used.
- Enable **CloudTrail** in all regions to log every IAM action.

- Set an account **password policy** (minimum length, complexity, rotation).
- Consider **IAM Identity Center** and permission boundaries for larger organisations.
- Rotate or eliminate any IAM user access keys; prefer roles and temporary credentials.

14. Conclusion

This phase delivered a structured IAM design for the 3-tier application. Three groups (Admins, Developers and Auditors) were created with custom least-privilege policies, administrators were forced to use MFA, and an IAM role allowed the EC2 application server to reach S3 without any hardcoded credentials. Testing confirmed that each identity can perform only its intended tasks and that sensitive actions are blocked by explicit denies. Together these controls reduce the risk of credential leakage, privilege escalation and insider misuse, and they complete the customer-side identity responsibilities identified in Phase 1.

Submitted by: Sumit Mhalaskar
PRN: 2124UCSM1065
Roll No.: 60
Course: B.Tech Cyber Security, Sanjivani University